

PRÁCTICA 1 — Propuesta del proyecto

Asignatura: Técnicas de Programación Avanzada (TPA) — Grupo A

Curso: 2026–2027

Temática elegida: 12. *Simulador de Bolsa y Carteras de Inversión*

Alumno: dani-18 (PRAC1 individual; la asignatura es en grupo para PRAC2–PRAC4)

> Documento maquetado en Markdown y exportado a PDF de forma automática
> (ver `tools/generar_propuesta_pdf.py`). La versión PDF es la que se
> entrega en el Campus Virtual; este `.md` es la fuente de verdad.

1. Problema

Construir un **simulador de mercado de valores con activos ficticios** sobre el que se podrán probar estrategias de inversión, medir métricas de rentabilidad/riesgo y registrar la evolución temporal de la cartera. La temática es la indicada en el enunciado de la asignatura (apartado 12):

- **Activos financieros ficticios.** Crear una taxonomía elemental (acciones, bonos, ETFs) y un sistema de cotizaciones determinista en función de una semilla y la volatilidad del activo.
- **Estrategias de inversión.** Como mínimo, una estrategia "Buy & Hold" y otra de aportación periódica (DCA), abstraídas más adelante (PRAC2) bajo un mismo contrato.
- **Evolución del mercado simulada.** El motor de mercado actualiza cotizaciones a partir de un movimiento browniano geométrico anualizado, produciendo una serie OHLCV por sesión.
- **Gestión de carteras.** El núcleo de la aplicación: saldo de efectivo, posiciones abiertas y un registro **inmutable** de operaciones ejecutadas (compra/venta con comisiones).
- **Métricas de rentabilidad y riesgo.** ROI total, drawdown máximo. Volatilidad anualizada y *Sharpe ratio* se completarán en PRAC4.

2. Alcance del proyecto

2.1 Alcance de la PRAC1 (lo que se entrega aquí)

La PRAC1 se centra en **Tema 1 — objetos, memoria y encapsulamiento**:

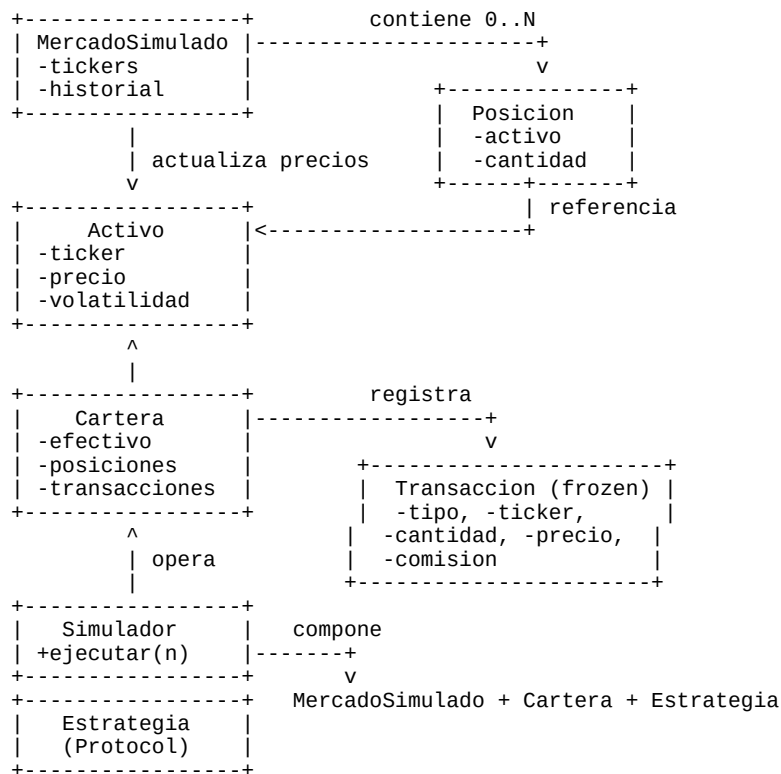
- Repositorio con `src/` layout y `pyproject.toml`.
- `python -m bolsa_sim --demo` imprime el flujo inicial (mercado → activo → cartera → 30 sesiones → resumen final).
- `python -m bolsa_sim --memoria` demuestra identidad vs igualdad, *aliasing* y el **defecto del argumento por defecto mutable** (identificado, explicado y corregido).
- Paquete **stdlib-only** para el código de producción (única dependencia externa permitida). Solo las herramientas de *test/dev* (pytest, black, ruff, reportlab, pdoc) se añaden como *optional-dependencies*.
- 77 *smoke* + unit + integración tests, cobertura $\approx 87\%$ (objetivo $\geq 70\%$ ya satisfecho desde la primera entrega).
- Skeleton de CI en GitHub Actions (black, ruff, pytest).

2.2 Roadmap (resumido)

Hito	Tema	Entregable principal
PRAC1	1	Modelo de dominio, encapsulamiento, memoria
PRAC2	2	Herencia, polimorfismo, excepciones, genericidad
PRAC3	3	Patrones (Strategy, Factory, Observer)
PRAC4	4	GUI, eventos, concurrencia + pulido + $\geq 70\%$ cov.

3. Modelo de dominio (PRAC1)

Diagrama de clases (texto, se representara tambien en el PDF):



Notas clave:

MercadoSimulado mantiene el mapping ticker -> Activo. Las Posicion lo referencian (asociacion, **no composicion**: el activo no se destruye al cerrar la posicion).

Cartera **no** conoce las estrategias; es Simulador quien compone mercado + cartera + estrategia.

Transaccion **no** se puede mutar (froze dataclass); el historial es append-only*.

Decisiones de diseño explícitas:

- Activo se escribe a mano (no @dataclass) para mostrar `__init__/@property/@setter/__repr__/@eq__/@hash__` (Tema 1).
- Posicion tiene `__hash__ = None` porque es un objeto mutable; se distingue, por tanto, de Transaccion que es **frozen**.
- Transaccion es @dataclass(frozen=True, slots=True); el historial es **append-only**, requisito del dominio (no se modifican operaciones pasadas).
- BarraDiaria es una typing.NamedTuple por ser un *dato compuesto de campos fijos e inmutable* (Tema 1, criterios tupla vs lista).
- Cartera._transacciones es **lista** (crece); las posiciones son un **diccionario** indexado por ticker (accesos O(1) y claves hashables).
- Estrategia se modela con un Protocol y duck typing; la jerarquía ABC y la genericidad llegan en PRAC2.

4. Modelo de memoria (Tema 1) — entregas clave

La asignatura exige **identificar, explicar y corregir** un defecto conceptual asociado al estado compartido. Lo hacemos en `src/bolsa_sim/memoria.py` y lo cubrimos en `tests/test_memoria.py`:

Fenómeno	Demostración en código
Identidad (<code>`is`</code>)	<code>`id(n) == id(10)`</code> en enteros singleton
Igualdad (<code>`==`</code>)	dos listas con idéntico contenido $\neq$ mismo objeto
Aliasing	<code>`b = a`</code> ; <code>`b.append(x)`</code> visible por <code>`a`</code>
Rebinding	<code>`t2 = (*t, x)`</code> produce nueva tupla
Hashable vs no	tuplas como claves de <code>`dict`</code> ; listas no
Defecto mutable default	<code>`registrar_evento_defectuoso`</code> comparte la lista
Corrección	<code>`registrar_evento_correcto`</code> usa <code>`None`</code> + lista nueva

Se ejecutan con `python -m bolsa_sim --memoria` y se validan en los tests.

5. Plan de las siguientes prácticas

- **PRAC2 (Tema 2)**. Convertir Estrategia en una jerarquía ABC con parámetros genéricos; introducir `TipologiaActivo` en la jerarquía de activos (`Accion`, `Bono`, `Etf`); excepciones de dominio (`OperacionInvalidaError`, etc.).
- **PRAC3 (Tema 3)**. Patrón Strategy formal; Factory para construir portafolios a partir de un perfil; Observer para suscribir el mercado a la GUI.
- **PRAC4 (Tema 4)**. Interfaz gráfica (Tkinter o Textual), eventos de teclado, concurrencia para correr varias simulaciones, integración con pandas para análisis y generación del PDF del informe final.

6. Cómo se ejecuta la PRAC1

```
# 1) entorno virtual
python -m venv .venv
source .venv/Scripts/activate          # en PowerShell: .venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"

# 2) tests + cobertura
python -m pytest --cov=bolsa_sim

# 3) calidad (PEP 8)
python -m black --check src tests
python -m ruff check src tests

# 4) demo "hello flow" (30 sesiones)
python -m bolsa_sim --demo

# 5) demo del Tema 1 (memoria)
python -m bolsa_sim --memoria

# 6) generar documentación de la API
python -m pdoc bolsa_sim -o docs/api

# 7) generar el PDF de la propuesta
python tools/generar_propuesta_pdf.py
```

7. Riesgos y dependencias

- `reportlab` se usa **solo** como herramienta de generación del PDF de propuesta, declarada en `[project.optional-dependencies]` y no se importa en ningún test.
- No hay dependencias en tiempo de ejecución que requiera instalar nada

extra (cumple "No se requieren dependencias externas: solo la biblioteca estándar").

- El generador de números pseudo-aleatorios (`random.Random`) se inyecta con semilla, lo que asegura simulaciones **reproducibles** (necesario para los tests).

PRAC1 — Técnicas de Programación Avanzada — Grupo A — curso 2026-2027.